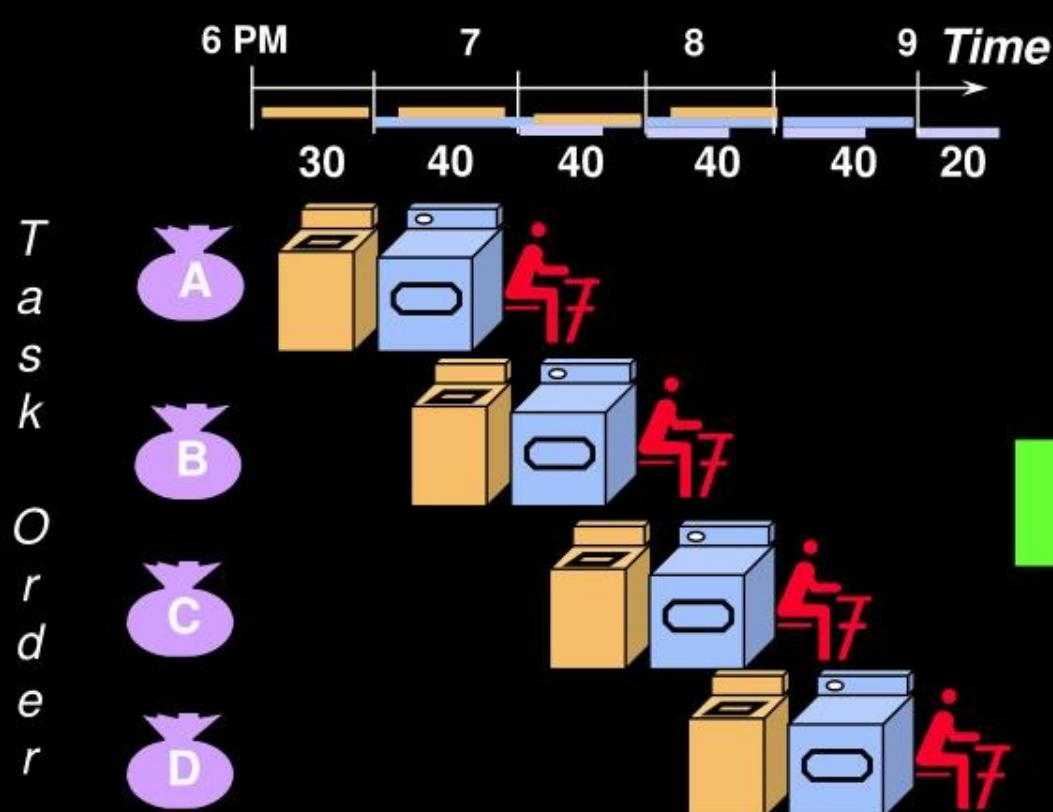


Lecture 7

Pipeline Hazards

Pipelining Lessons

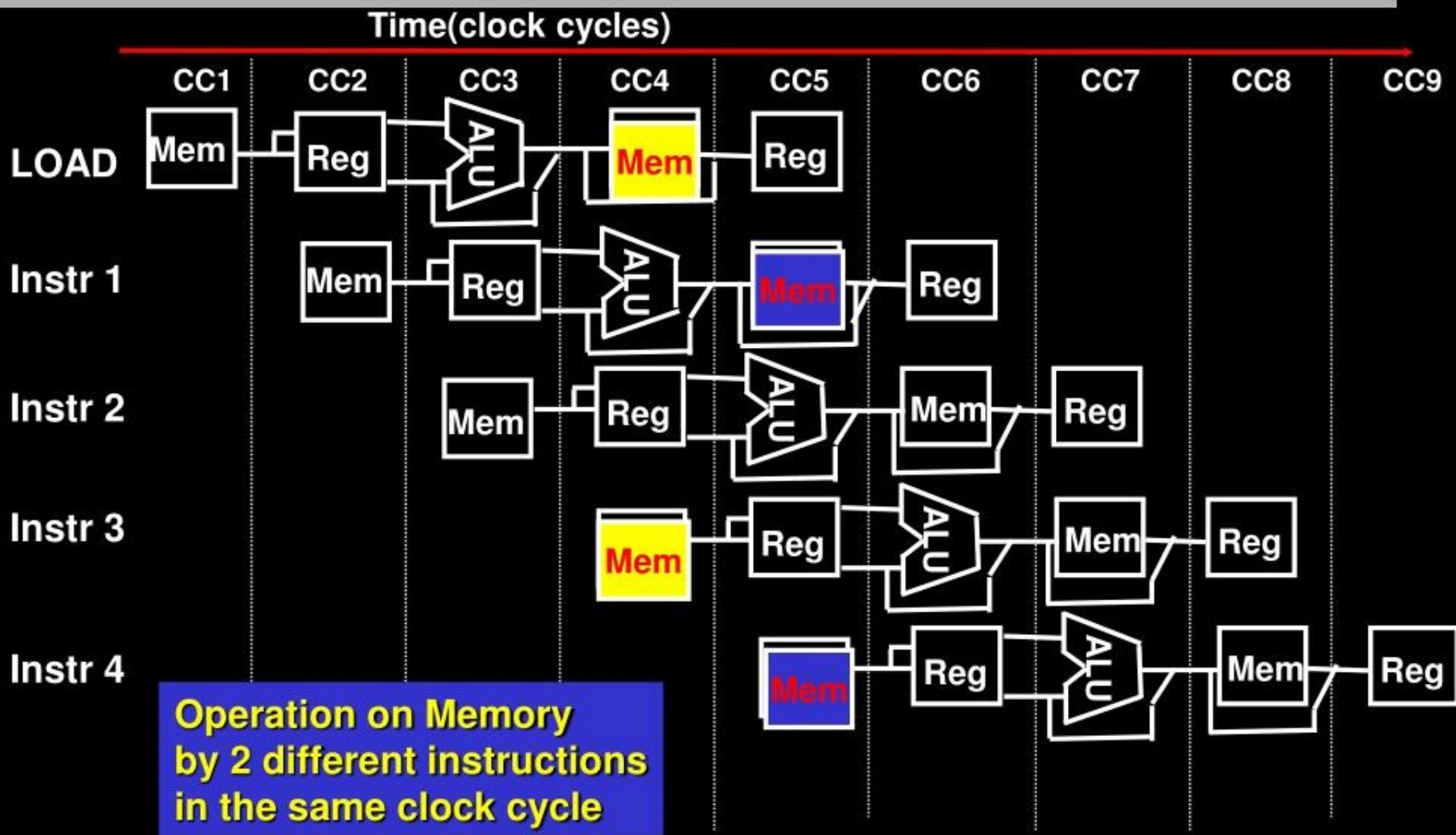


- **Pipelining doesn't help** latency of single task, it helps throughput of entire **workload**
- **Pipeline rate limited by** slowest pipeline stage
- Multiple tasks operating **simultaneously**
- **Potential speedup = Number pipe stages**
- **Unbalanced lengths of pipe stages reduces speedup**
- **Time to "fill" pipeline and time to "drain" it reduces speedup**

Its Not That Easy to Achieve the Promised Performance

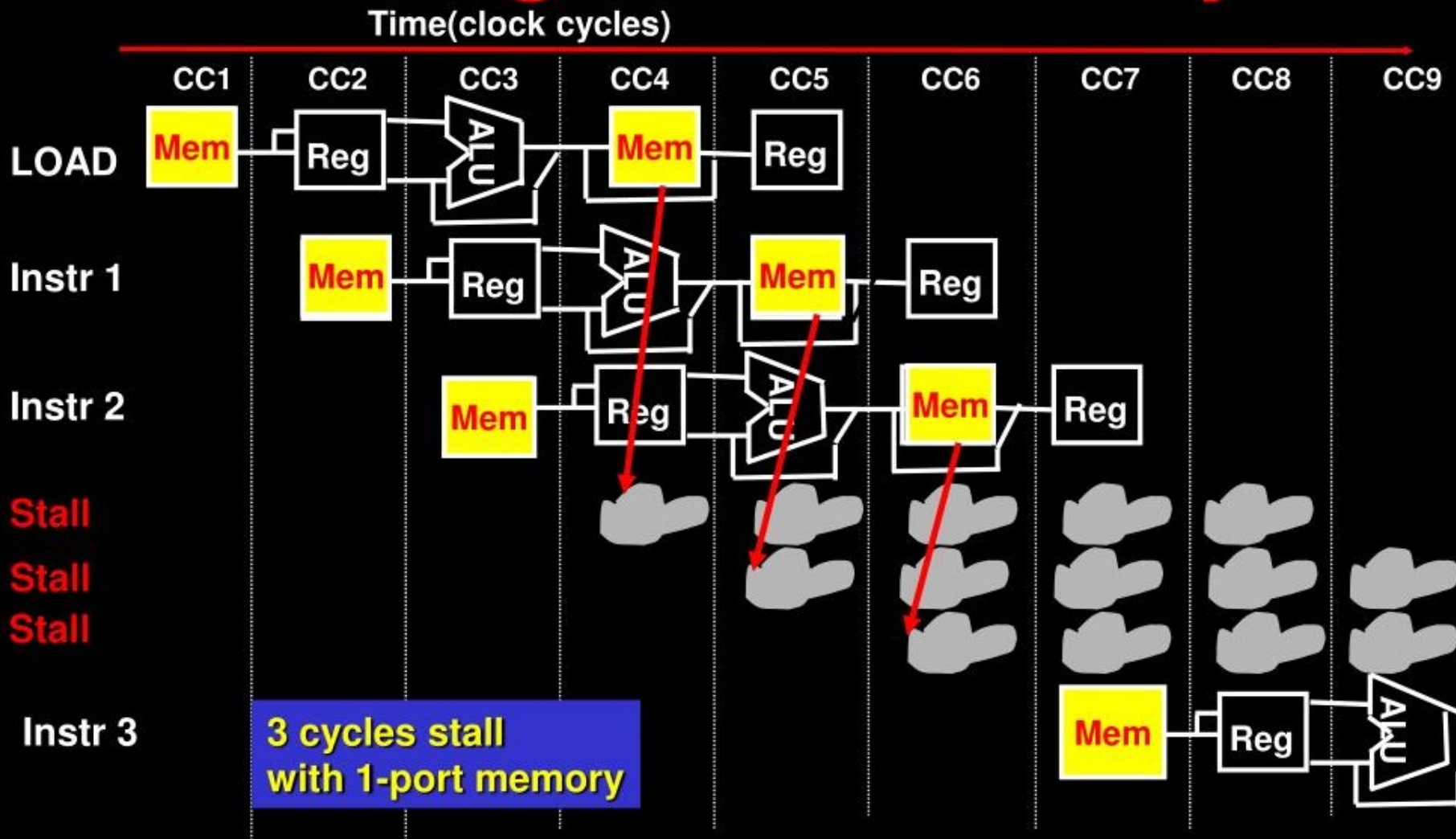
- **Limits to pipelining:** Hazards prevent the next instruction from executing during its designated clock cycle
 - **Structural hazards:** HW cannot support this combination of instructions
 - **Data hazards:** Instruction depends on result of prior instruction still in the pipeline
 - **Control hazards:** Pipelining of branches and other instructions that change the PC
- **Common solution is to stall the pipeline until the hazard is resolved, inserting one or more “bubbles”, i.e., idle clock cycles, in the pipeline**

Structural Hazards /Memory



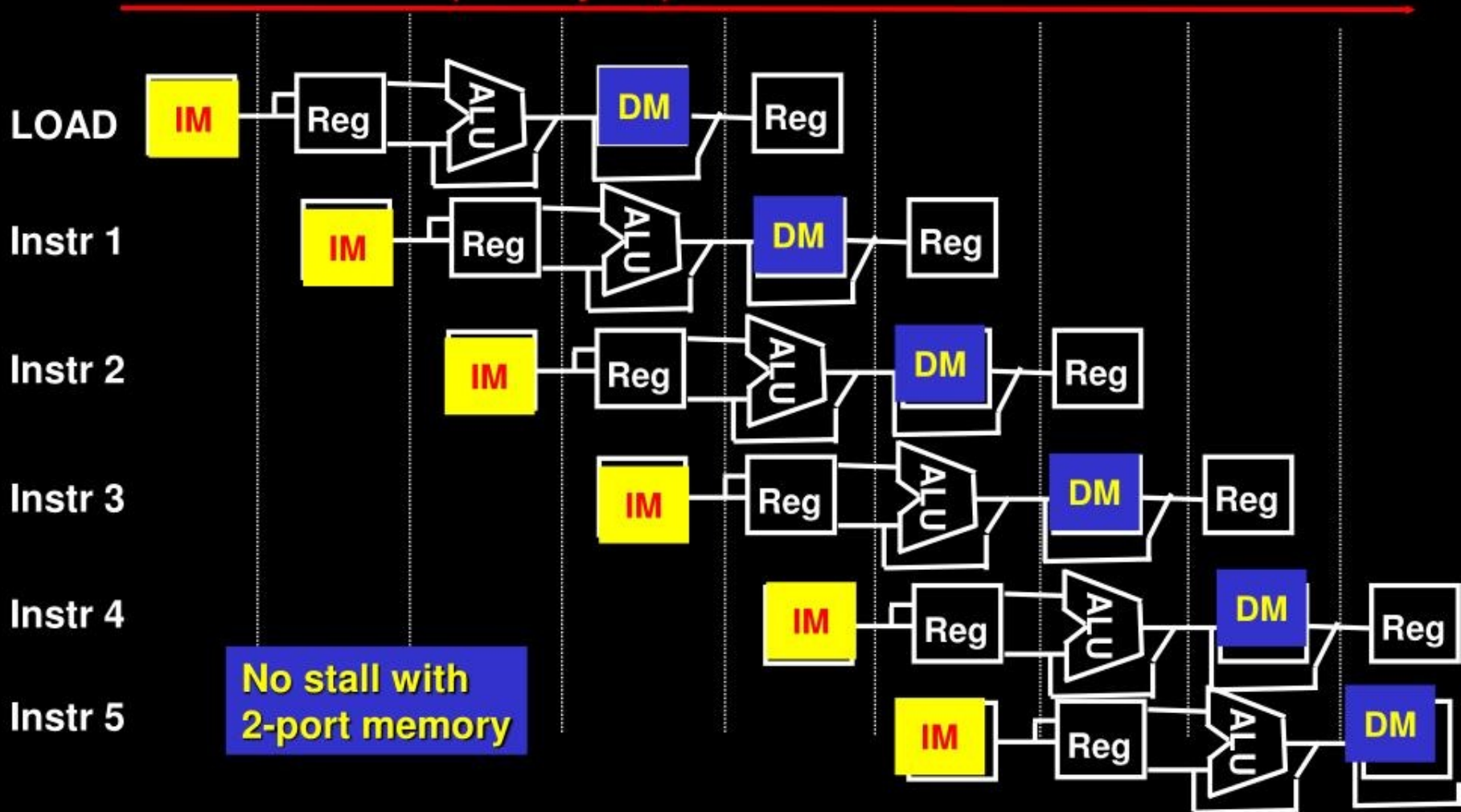
Structural Hazards with Single-Port Memory

Instruction Order



Avoiding Structural Hazard with Dual-Port Memory

Time(clock cycles)



Speed Up Equation for Pipelining

$$\begin{aligned}\text{Speedup from pipelining} &= \frac{\text{Ave Instr Time}_{\text{unpipelined}}}{\text{Ave Instr Time}_{\text{pipelined}}} \\ &= \frac{\text{CPI}_{\text{unpipelined}} \times \text{Clock Cycle}_{\text{unpipelined}}}{\text{CPI}_{\text{pipelined}} \times \text{Clock Cycle}_{\text{pipelined}}} \\ &= \frac{\text{CPI}_{\text{unpipelined}}}{\text{CPI}_{\text{pipelined}}} \times \frac{\text{Clock Cycle}_{\text{unpipelined}}}{\text{Clock Cycle}_{\text{pipelined}}}\end{aligned}$$

$$\text{Ideal CPI} = \text{CPI}_{\text{unpipelined}} / \text{Pipeline depth (Number of pipeline stages)}$$

$$\text{Speedup} = \frac{\text{Ideal CPI} \times \text{Pipeline depth}}{\text{CPI}_{\text{pipelined}}} \times \frac{\text{Clock Cycle}_{\text{unpipelined}}}{\text{Clock Cycle}_{\text{pipelined}}}$$

Ideal CPI for pipelined machines is almost always 1

Speed Up Equation for Pipelining

$$\begin{aligned}\text{CPI}_{\text{pipelined}} &= \text{Ideal CPI} + \text{Pipeline stall clock cycles per instr} \\ &= 1 + \text{Pipeline stall clock cycles per instr}\end{aligned}$$

$$\text{Speedup} = \frac{\text{Ideal CPI} \times \text{Pipeline depth}}{\text{Ideal CPI} + \text{Pipeline stall CPI}} \times \frac{\text{Clock Cycle}_{\text{unpipelined}}}{\text{Clock Cycle}_{\text{pipelined}}}$$

$$\text{Speedup} = \frac{\text{Pipeline depth}}{1 + \text{Pipeline stall CPI}} \times \frac{\text{Clock Cycle}_{\text{unpipelined}}}{\text{Clock Cycle}_{\text{pipelined}}}$$

Dual-Port vs Single-Port Memory

- Machine A: 2-port memory(needs no stall for Load); same clock cycle as unpipelined machine
- Machine B: 1-ported memory(needs 3 cycles stall for Load); 1.05 times faster clock rate than the unpipelined machine
- Ideal CPI = 1 for both
- Loads are 40% of instructions executed

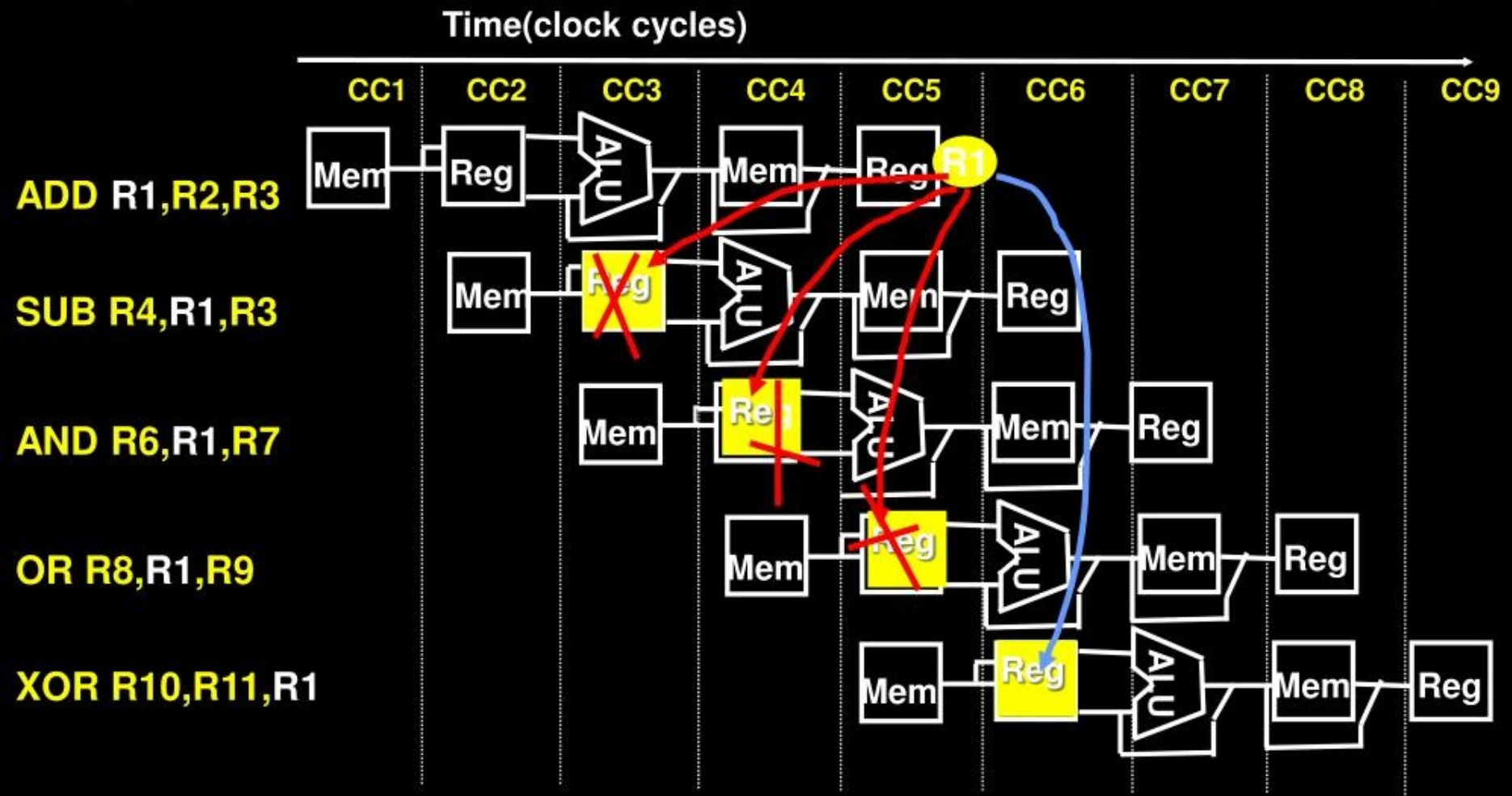
$$\text{SpeedUp}_A = [\text{Pipeline Depth}/(1 + 0)] \times (\text{clock}_{\text{unpipe}}/\text{clock}_{\text{pipe}}) \\ = \text{Pipeline Depth}$$

$$\text{SpeedUp}_B = \text{Pipeline Depth}/(1 + 0.4 \times 3) \times (\text{clock}_{\text{unpipe}}/(\text{clock}_{\text{unpipe}}/1.05)) \\ = (\text{Pipeline Depth}/1.2) \times 1.05 \\ = 0.87 \times \text{Pipeline Depth}$$

$$\text{SpeedUp}_A / \text{SpeedUp}_B = \text{Pipeline Depth}/(0.87 \times \text{Pipeline Depth}) = 1.15$$

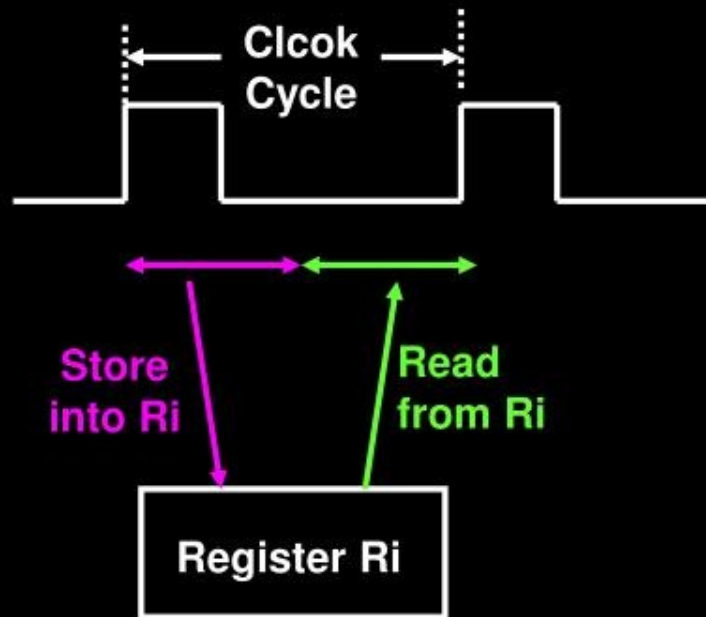
Machine A is 1.15 times faster

Data Hazard on Registers

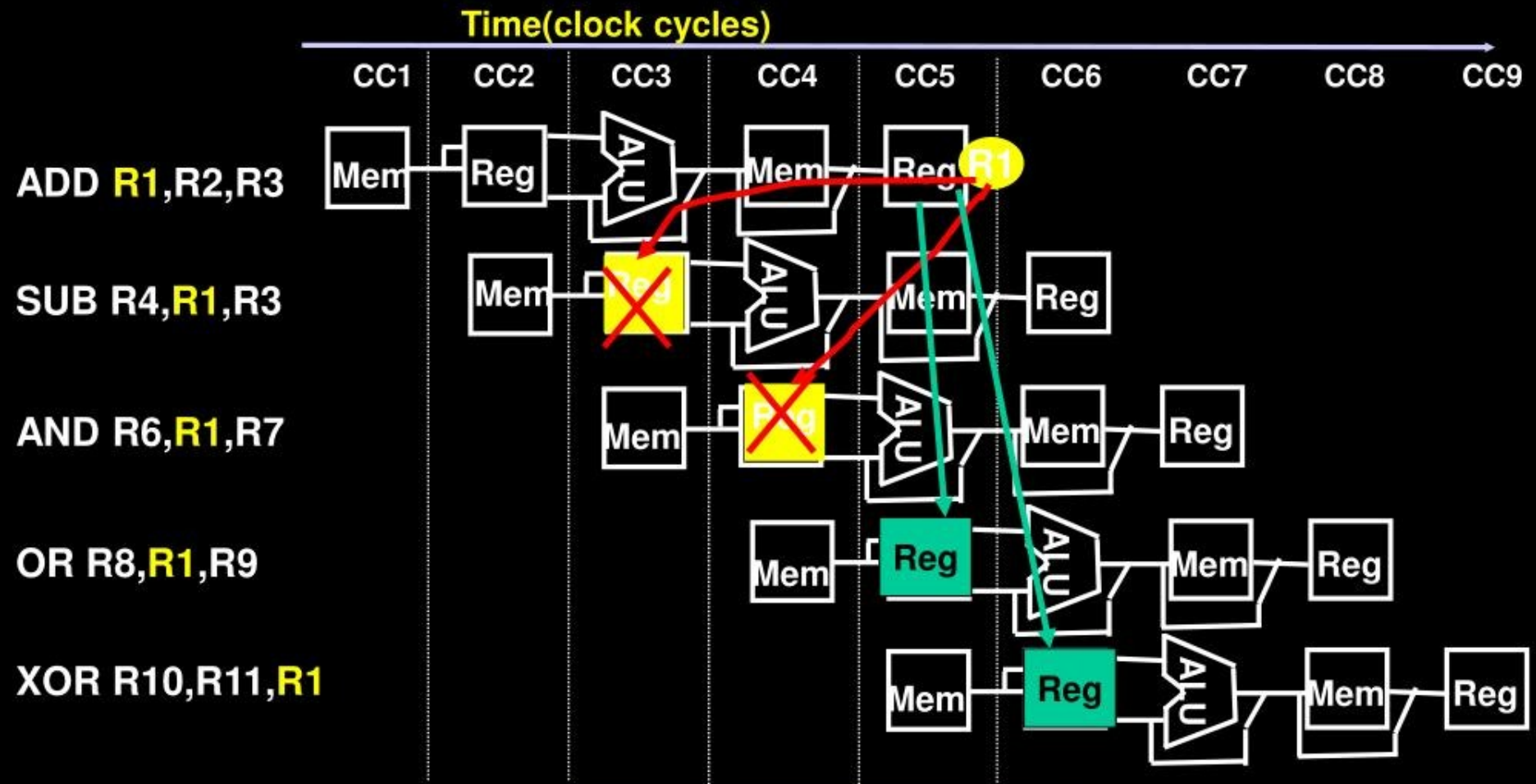


Data Hazard on Registers

Registers can be made to read and store in the same cycle such that data is stored in the first half of the clock cycle, and that data can be read in the second half of the same clock cycle



Data Hazard on Registers



Three Generic Data Hazards

Instr_i followed by Instr_j

Read After Write (RAW)

Instr_j tries to read operand before Instr_i writes it

Instr_i	LW	R1, 0(R2)
Instr_j	SUBR	4, R1, R5

Three Generic Data Hazards

Instr_i followed by Instr_j

- **Write After Read (WAR)**
Instr_j tries to write operand before Instr_i reads it

Instr_i	ADD	R1, R2, R3
Instr_j	LW	R2, 0(R5)

Can't happen in DLX 5 stage pipeline because:

- **All instructions take 5 stages,**
- **Reads are always in stage 2, and**
- **Writes are always in stage 5**

Three Generic Data Hazards

Instr_i followed by Instr_j

Write After Write (WAW)

Instr_j tries to write operand before Instr_i writes it

- Leaves wrong result (Instr_i not Instr_j)

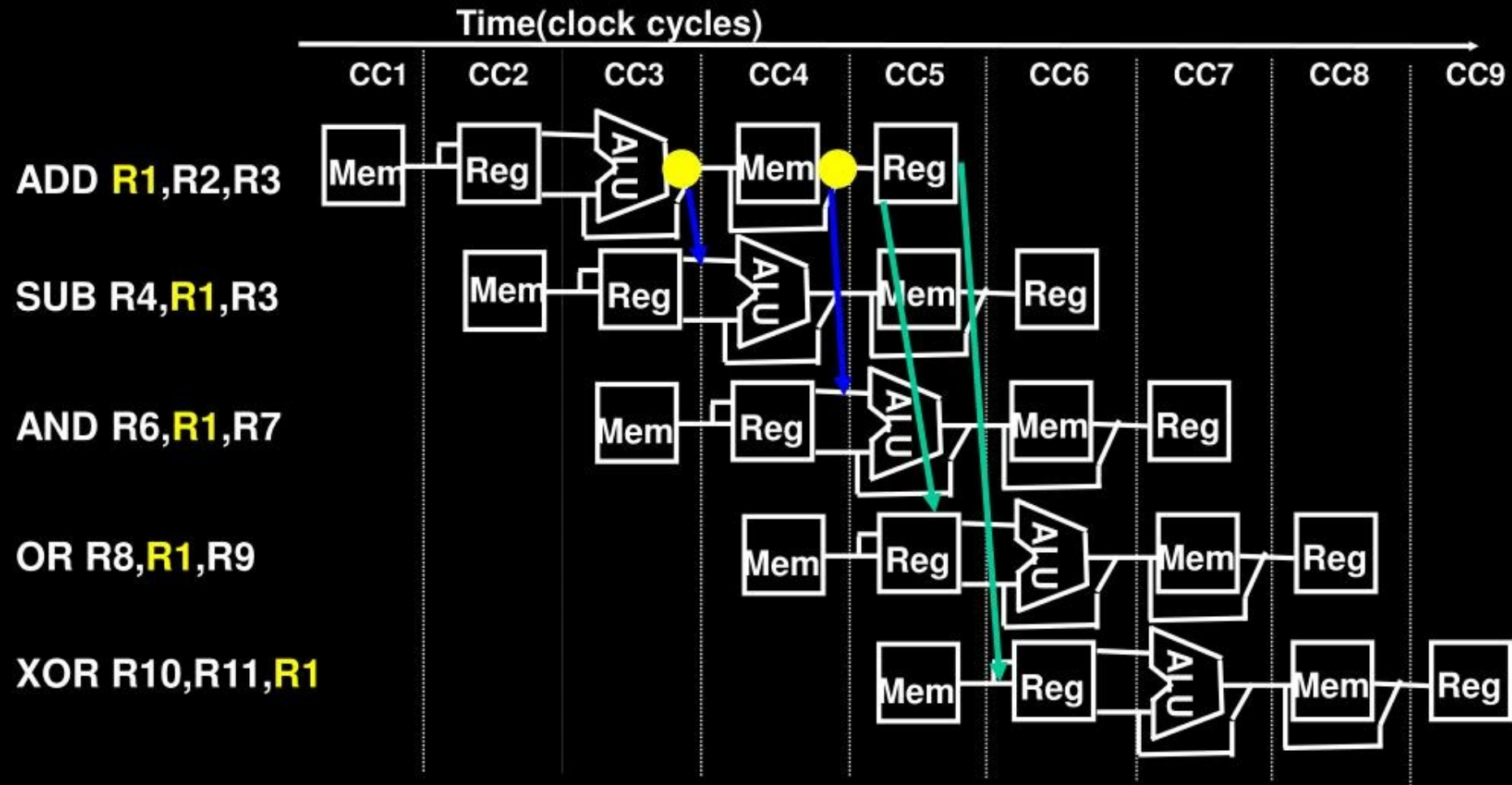
Instr_i	LW	$\text{R1}, 0(\text{R2})$
Instr_j	LW	$\text{R1}, 0(\text{R3})$

Can't happen in DLX 5 stage pipeline because:

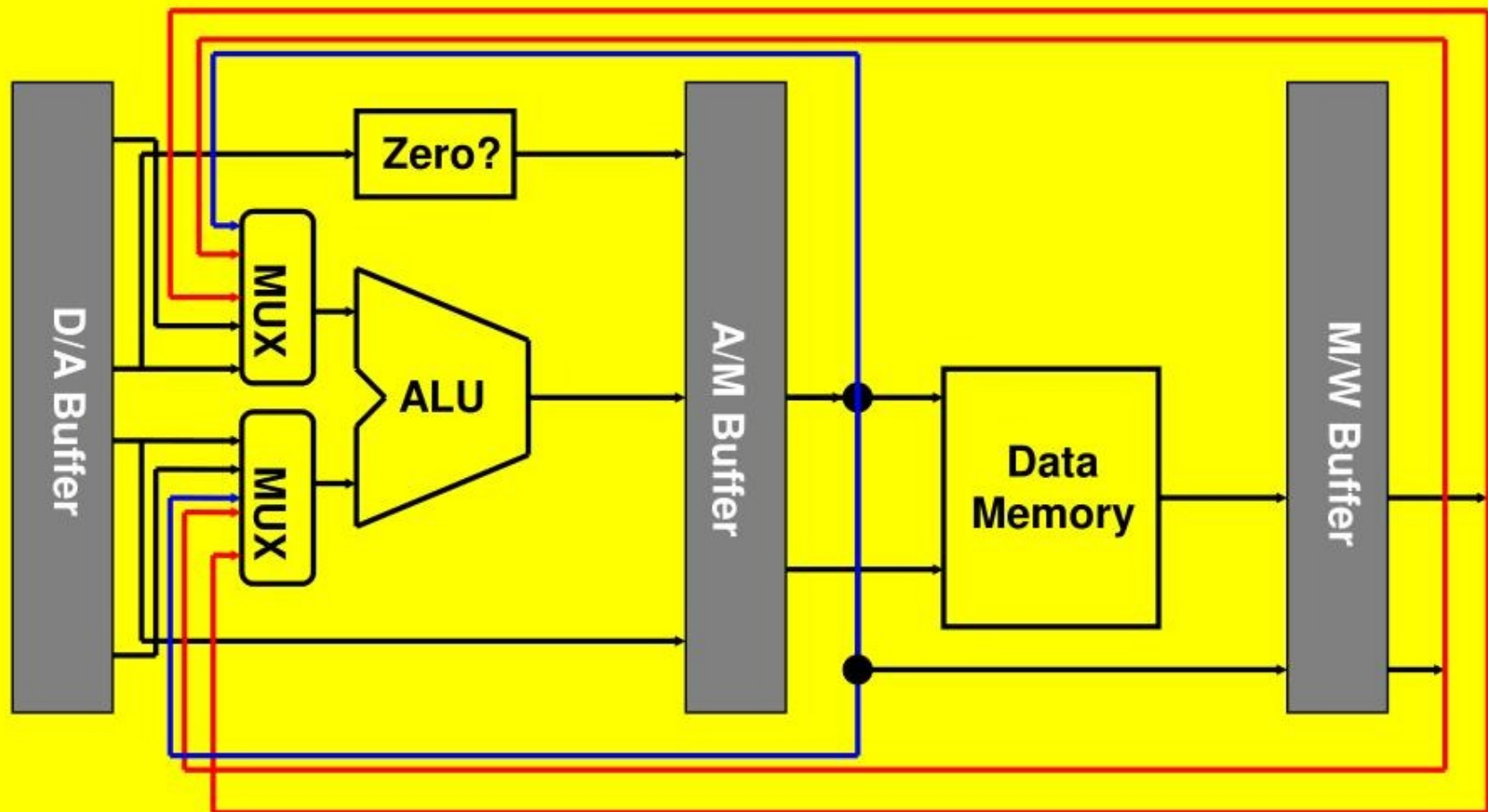
- All instructions take 5 stages, and
- Writes are always in stage 5

Will see WAR and WAW in later more complicated pipes

Forwarding to Avoid Data Hazards



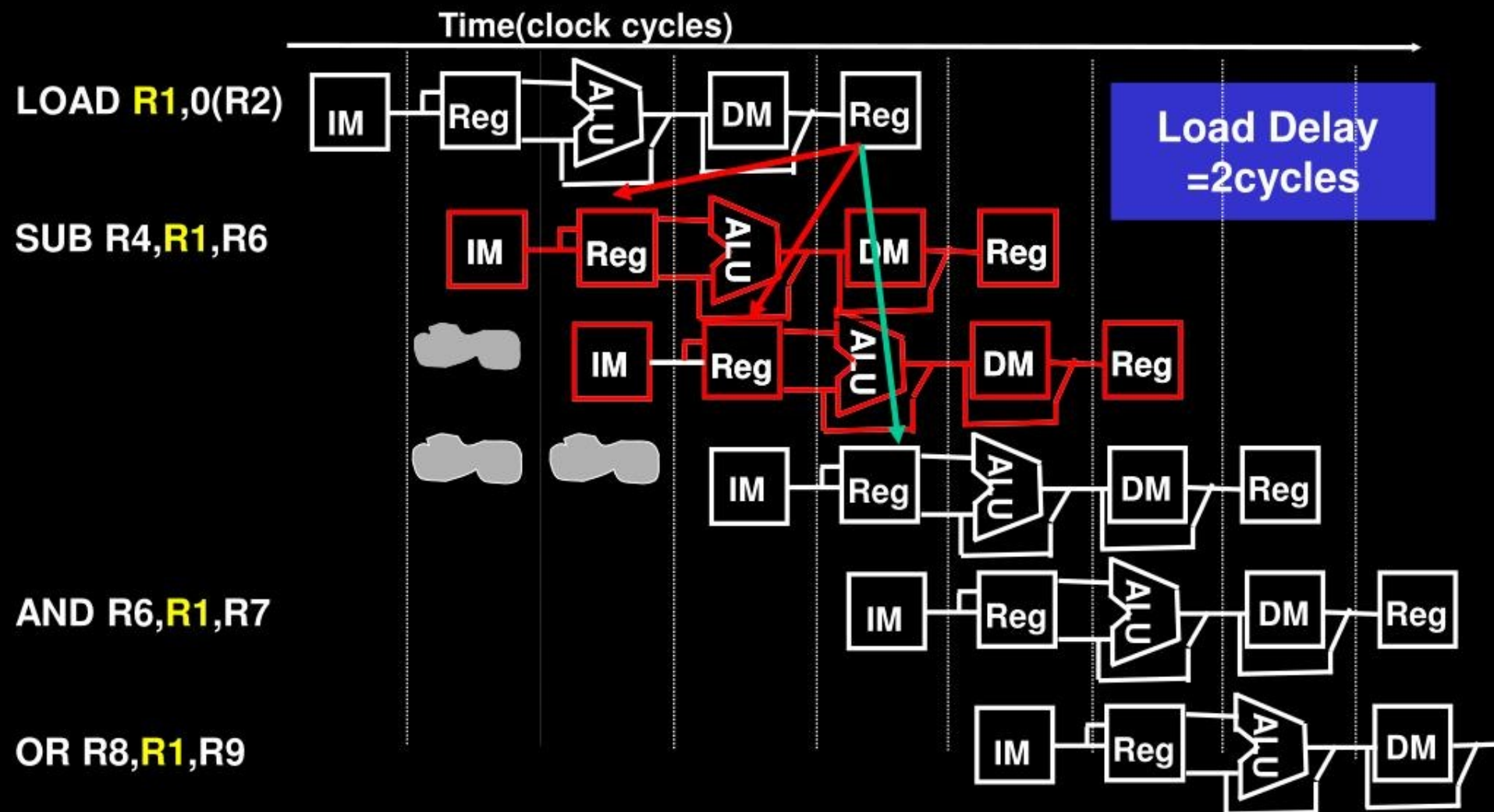
HW Change for Forwarding





Private Property
Lodge guests
patrons only
DANGER

Load Delay Due to Data Hazard



Load Delay with Forwarding

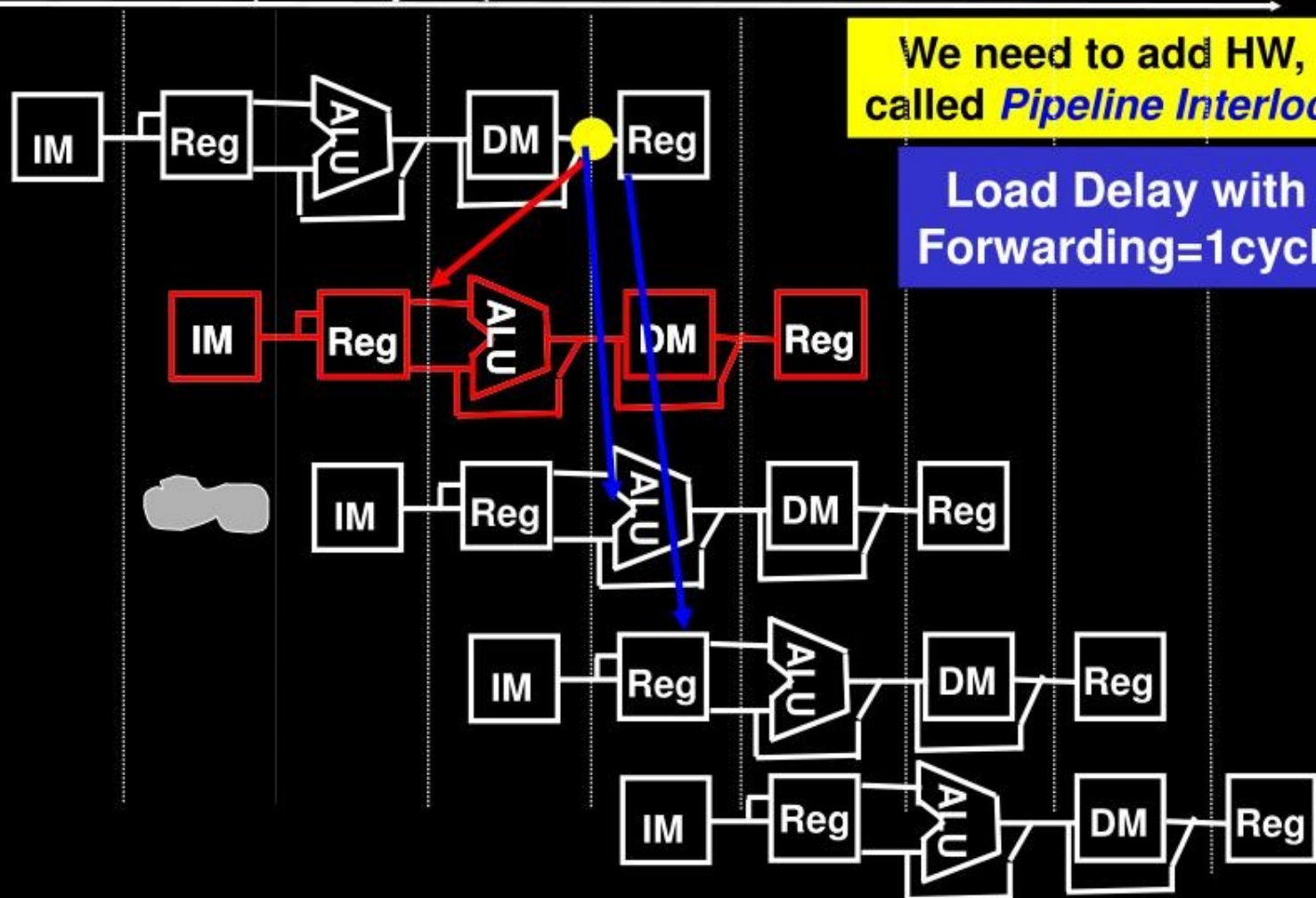
Time(clock cycles) →

LOAD R1,0(R2)

SUB R4,R1,R6

AND R6,R1,R7

OR R8,R1,R9



Software Scheduling to Avoid Load Hazards

Try to produce fast code for

$a = b + c;$

$d = e - f;$

assuming $a, b, c, d, e,$ and f are in memory.

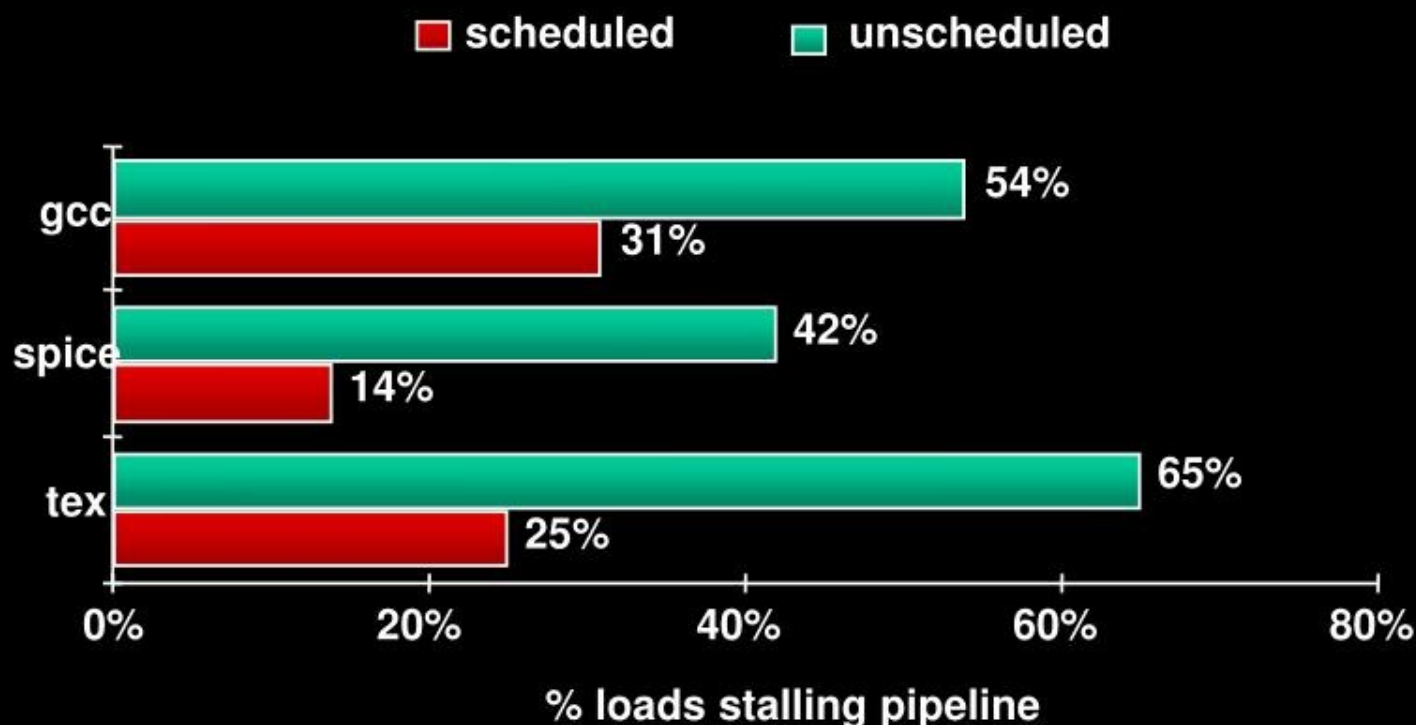
Slow code(with forwarding):

	LW	Rb,b	
	LW	Rc,c	
RAW	ADD	Ra,Rb,Rc	Stall
RAW	SW	a,Ra	Stall
	LW	Re,e	
	LW	Rf,f	
RAW	SUB	Rd,Re,Rf	Stall
RAW	SW	d,Rd	Stall

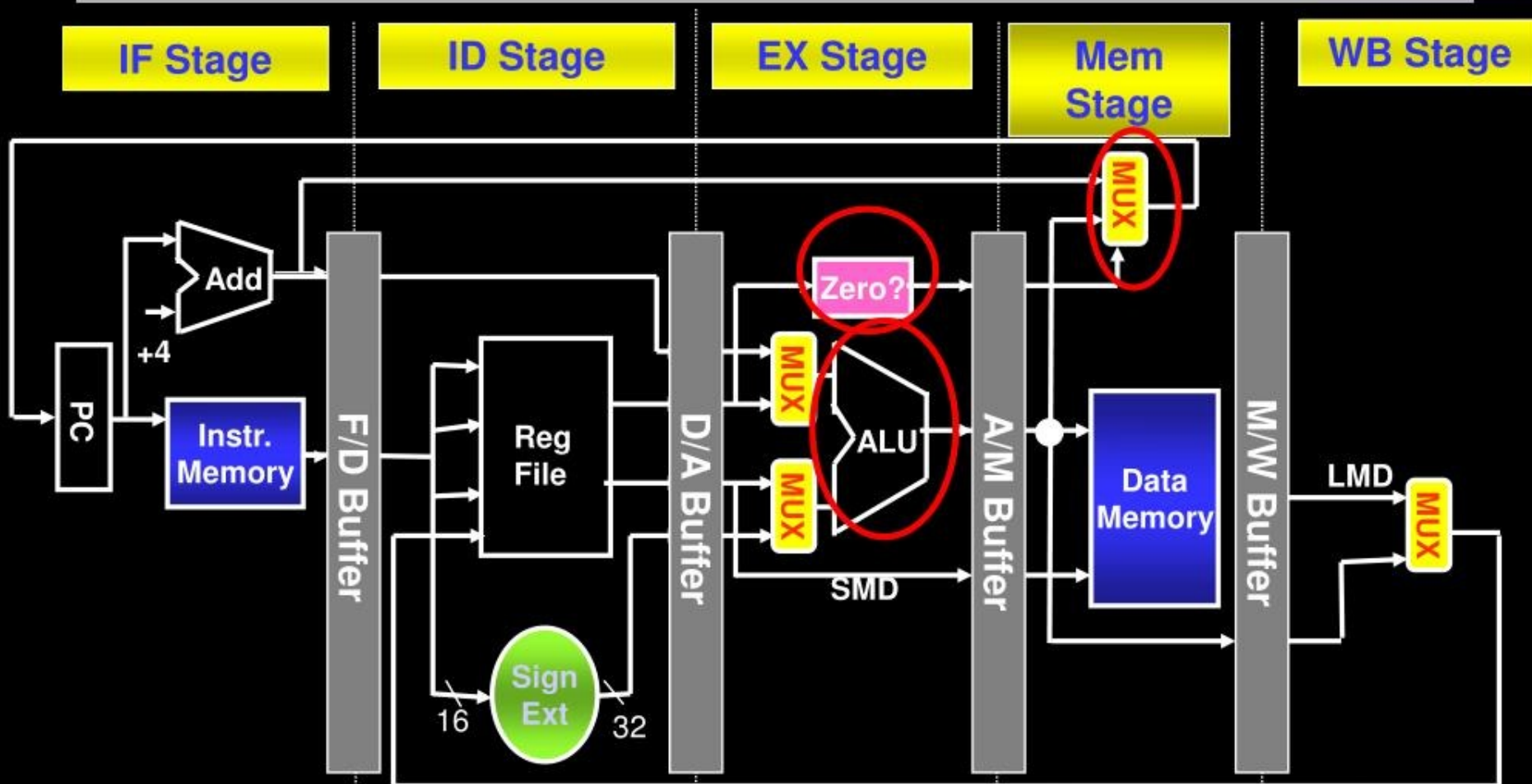
Fast code:

	LW	Rb,b	
	LW	Rc,c	
	LW	Re,e	
	ADD	Ra,Rb,Rc	
	LW	Rf,f	
	SW	a,Ra	
	SUB	Rd,Re,Rf	
RAW	SW	d,Rd	Stall

Compiler Avoiding Load Stalls



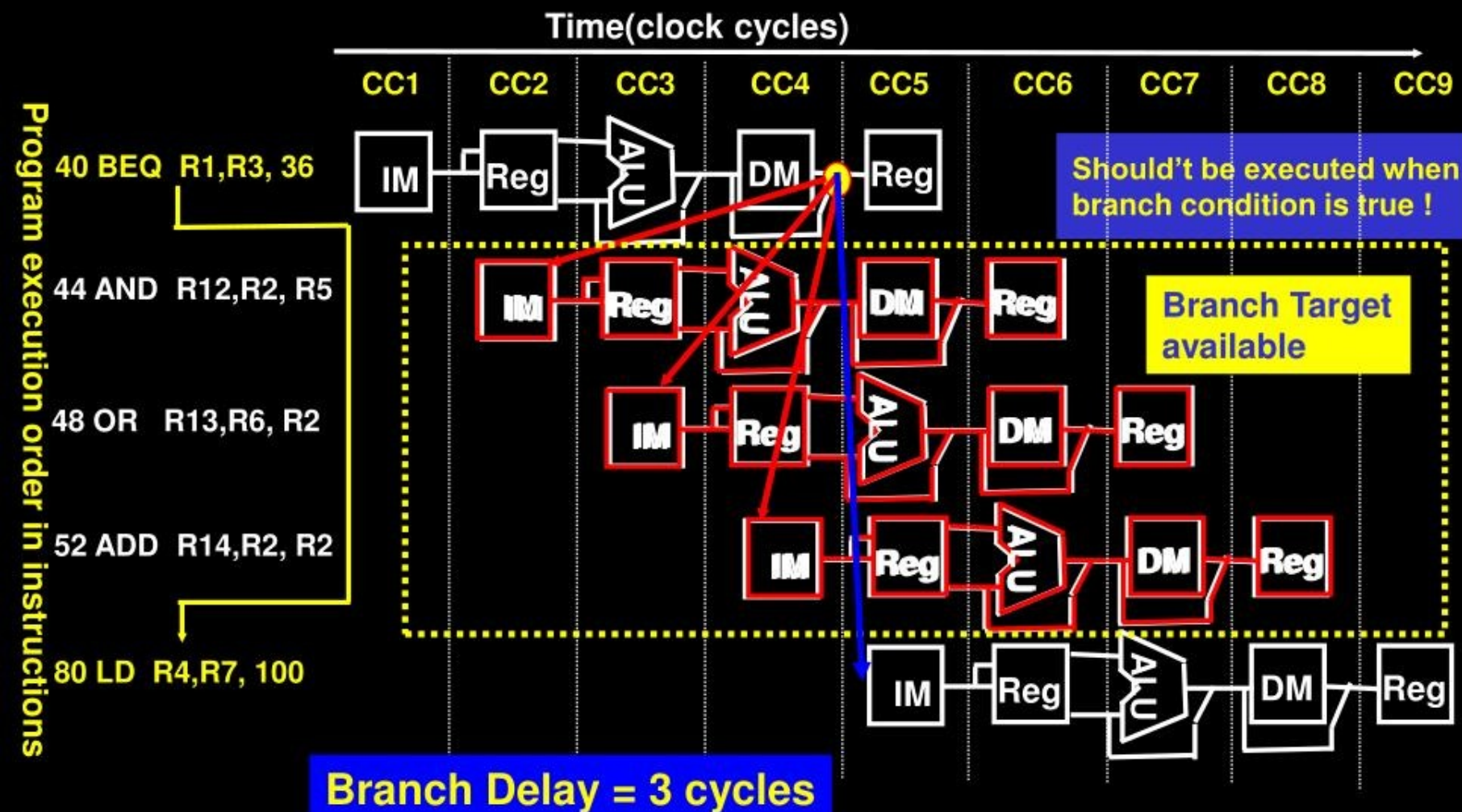
Pipelined DLX Datapath



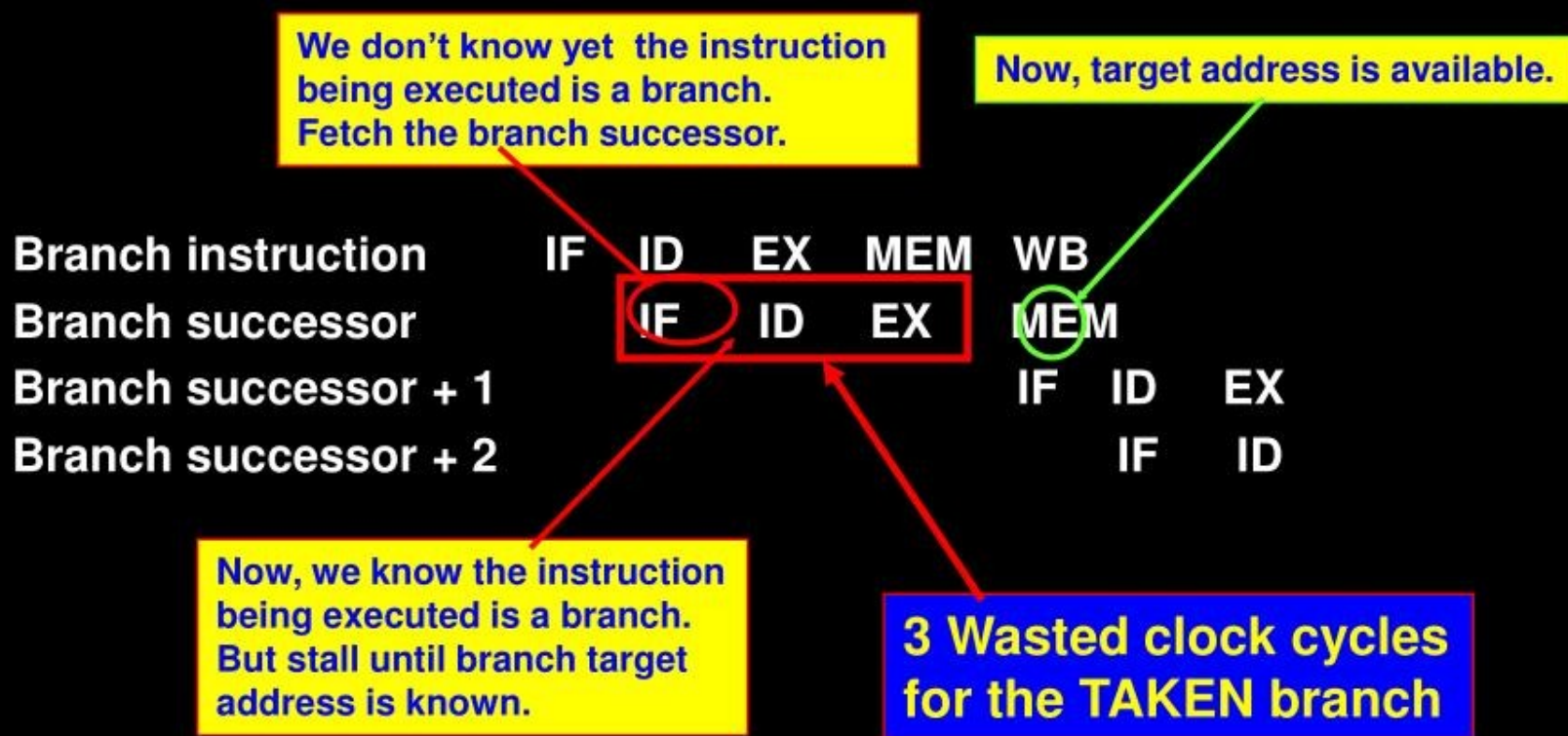
- Branch Address Calculation
- Decide Condition

- Branch Decision for target address

Control Hazard on Branches: Three Stall Cycles



Control Hazard on Branches: Three Stall Cycles



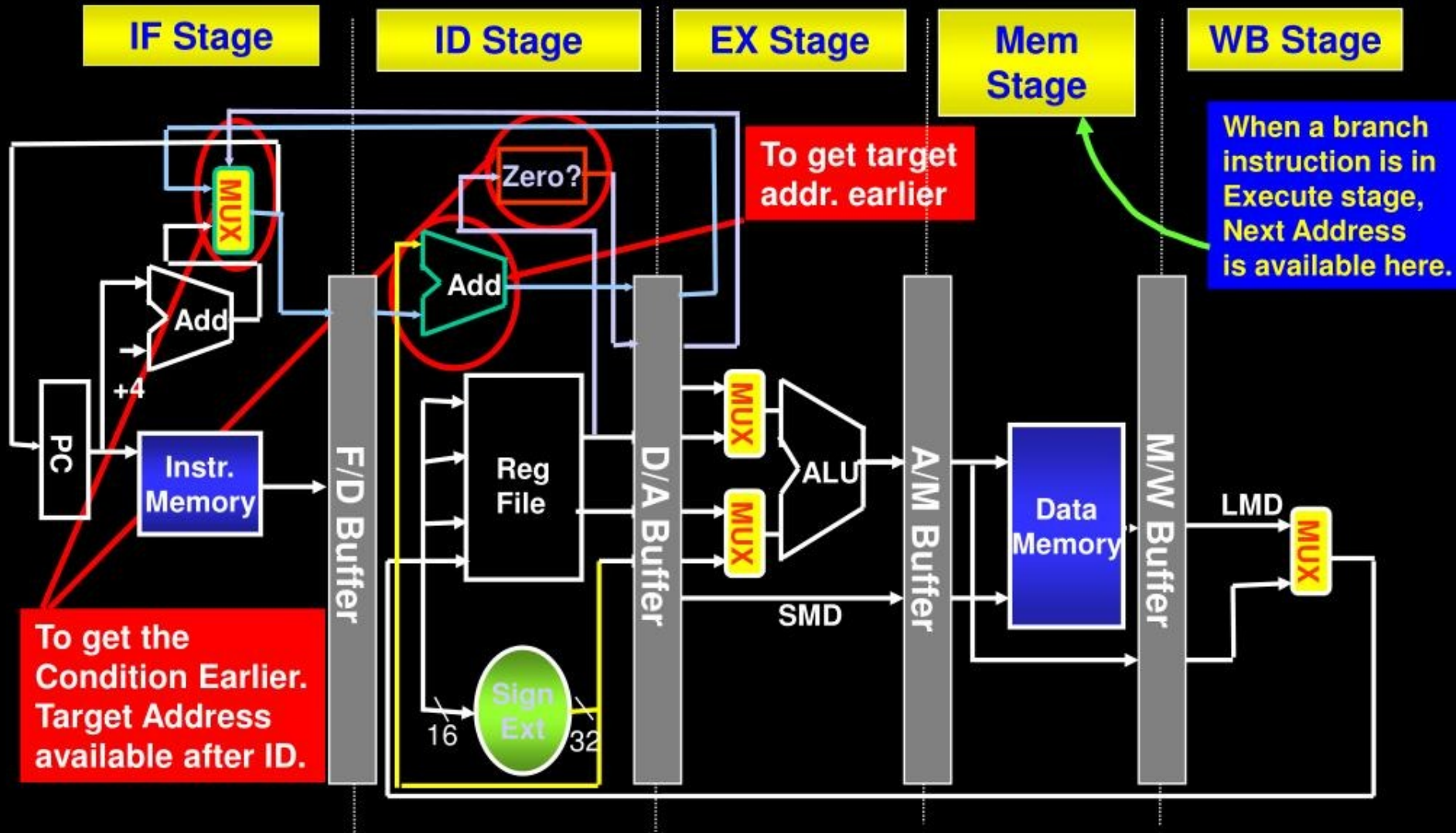
Branch Stall Impact

- If $CPI = 1$, 30% branch, Stall 3 cycles $\Rightarrow$ new $CPI = 1.9$
 - Half of the ideal speed
- Two part solution:
 - Determine the branch is **TAKEN** or **NOT TAKEN** sooner, **AND**
 - Compute **TAKEN Branch Address**(Branch Target) earlier
- **DLX** branch tests if register = 0 or 1

DLX Solution: Get New PC earlier

- Move **Zero test to ID stage**
- **Additional ADDER** to calculate New PC(taken PC) in ID stage
- 1 clock cycle penalty for branch in contrast to 3 cycles

Pipelined DLX Datapath





Branch Behavior in Programs

- **Conditional branch frequencies**
 - integer average --- 14 to 16 %
 - floating point --- 3 to 12 %
- **Forward and backward taken branches**
 - forward taken --- 60 %
 - backward taken --- 85 %
 - the average of all conditional branches ---- 67 %

4 Branch Hazard Alternatives

- Stall until branch direction is clear
- Predict branch NOT TAKEN
- Predict branch TAKEN
- Delayed branch

4 Branch Hazard Alternatives:

(1) STALL

Stall until branch direction is clear

Branch instruction	IF	ID	EX	MEM	WB				
Branch successor		stall	stall	stall	IF	ID	EX	MEM	
Branch successor + 1						IF	ID	EX	
Branch successor + 2							IF	ID	

3 cycle penalty

Revised DLX pipeline(get the branch address at EX)

Branch instruction	IF	ID	EX	MEM	WB				
Branch successor		stall	IF	ID	EX	MEM	WB		
Branch successor + 1				IF	ID	EX	MEM		
Branch successor + 2					IF	ID			

1 cycle penalty(Branch Delay Slot)

4 Branch Hazard Alternatives:

(2) Predict Branch “NOT TAKEN”

- Execute successor instructions in the sequence
- PC+4 is already calculated, so use it to get the next instruction
- Flush instructions in the pipeline if branch is actually TAKEN
- Advantage of late pipeline state update
- 47% of DLX branches are NOT TAKEN on the average

NOT TAKEN branch instruction i	IF	ID	EX	MEM	WB		
instruction i+1		IF	ID	EX	MEM	WB	
instruction i+2			IF	ID	EX	MEM	WB

No
penalty

TAKEN branch instruction i	IF	ID	EX	MEM	WB		
instruction i+1		IF	ID	EX	MEM	WB	
instruction T			IF	ID	EX	MEM	WB

1 cycle
penalty

Flush this instruction in progress

4 Branch Hazard Alternatives: (3) Predict Branch "TAKEN"

- 53% DLX branches TAKEN on average
- Branch target address available after ID in DLX
 - DLX still incurs 1 cycle branch penalty for TAKEN branch
 - Other machines: branch target known before outcome

TAKEN address not available at this time



2 cycle penalty in DLX(1 in other machines).

TAKEN address available



1 cycle penalty in DLX(0 in other machines)

4 Branch Hazard Alternatives: (4) Delayed Branch

Delayed Branch

- Delay branch to take place **AFTER** a successor instruction

branch instruction

sequential successor₁

sequential successor₂

.....

sequential successor_n

branch target if taken



Delayed Branch of length n

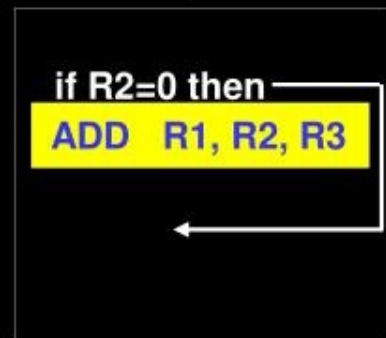
- 1 slot delayed branch allows proper decision and branch target address in 5 stage DLX pipeline with control hazard improvement

Delayed Branch

- **Where to get instructions to fill branch delay slot?**
 - Before branch instruction
 - From the target address: only valuable when branch TAKEN
 - From fall through: only valuable when branch NOT TAKEN
 - Canceling branches allow more slots to be filled
- **Compiler effectiveness for single delayed branch slot:**
 - Fills about 60% of delayed branch slots
 - About 80% of instructions executed in delayed branch slots are useful in computation
 - About 50% ($60\% \times 80\%$) of slots usefully filled

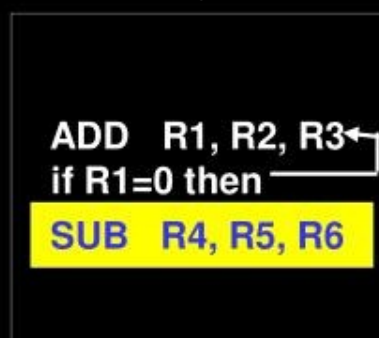
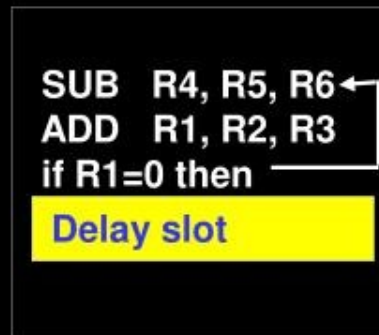
4 Branch Hazard Alternatives: Delayed Branch

From before



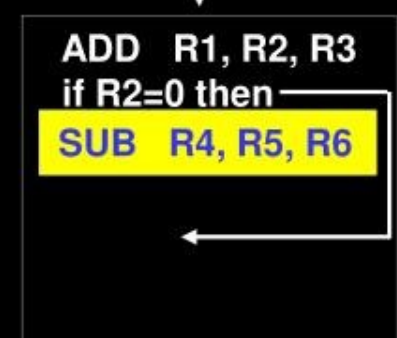
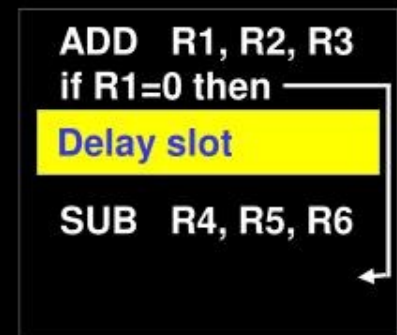
- Always improve performance
- Branch must not depend on rescheduled instructions

From target



- Improve performance when TAKEN(loop)
- Must be alright to execute rescheduled instructions if Not Taken
- May need duplicate the instruction if it is the target of another branch instr.

From fall through



- Improve performance when NOT TAKEN
- Must be alright to execute instructions of Taken

Limitations on Delayed Branch

- **Difficulty in finding useful instructions to fill the delayed branch slots**
- **Solution - Squashing**
 - Delayed branch associated with a branch prediction
 - Instructions in the predicted path are executed in the delayed branch slot
 - If the branch outcome is mispredicted, instructions in the delayed branch slot are squashed(discarded)

Canceling Branch

- **Used when the delayed branch scheduling, i.e., filling the delay slot cannot be done due to**
 - **Restrictions on scheduling instructions at the delay slots**
 - **Limitations on the ability to predict whether it will TAKE or NOT TAKE at compile time**
- **Instruction includes the direction that the branch was predicted**
 - **When the branch behaves as predicted, the instructions in the delay slot are executed**
 - **When branch is incorrectly predicted, the instructions in the delay slot are turned into No-OPs**
- **Canceling Branch allows to fill the delay slot even if the instruction to be filled in the delay slot does not meet the requirements**

Evaluating Branch Alternatives

$$\begin{aligned} \text{Pipeline speedup} &= \text{Pipeline depth} / \text{CPI} \\ &= \frac{\text{Pipeline depth}}{1 + \text{Branch frequency} \times \text{Branch penalty}} \end{aligned}$$

Conditional and Unconditional collectively 14% frequency,
65% of branch is TAKEN

<i>Scheduling scheme</i>	<i>Branch penalty</i>	<i>CPI</i>	<i>speedup vs unpipelined</i>	<i>speedup vs stall</i>
Stall pipeline	3	$1 + 0.14 \times 3 = 1.42$	$5 / 1.42 = 3.5$	1.0
Predict Taken	1	$1 + 0.14 \times 1 = 1.14$	$5 / 1.14 = 4.4$	1.26
Predict Not Taken	1	$1 + 0.14 \times 0.65 = 1.09$	$5 / 1.09 = 4.5$	1.29
Delayed branch	0.5	$1 + 0.14 \times 0.5 = 1.07$	$5 / 1.07 = 4.6$	1.31

Static(Compiler) Prediction of Taken/Untaken Branches

Code Motion

LW ← R1, 0(R2)
SUB R1, R1, R3
BEQZ R1, L
OR R4, R5, R6
ADD R10, R4, R3
L: ADD R7, R8, R9

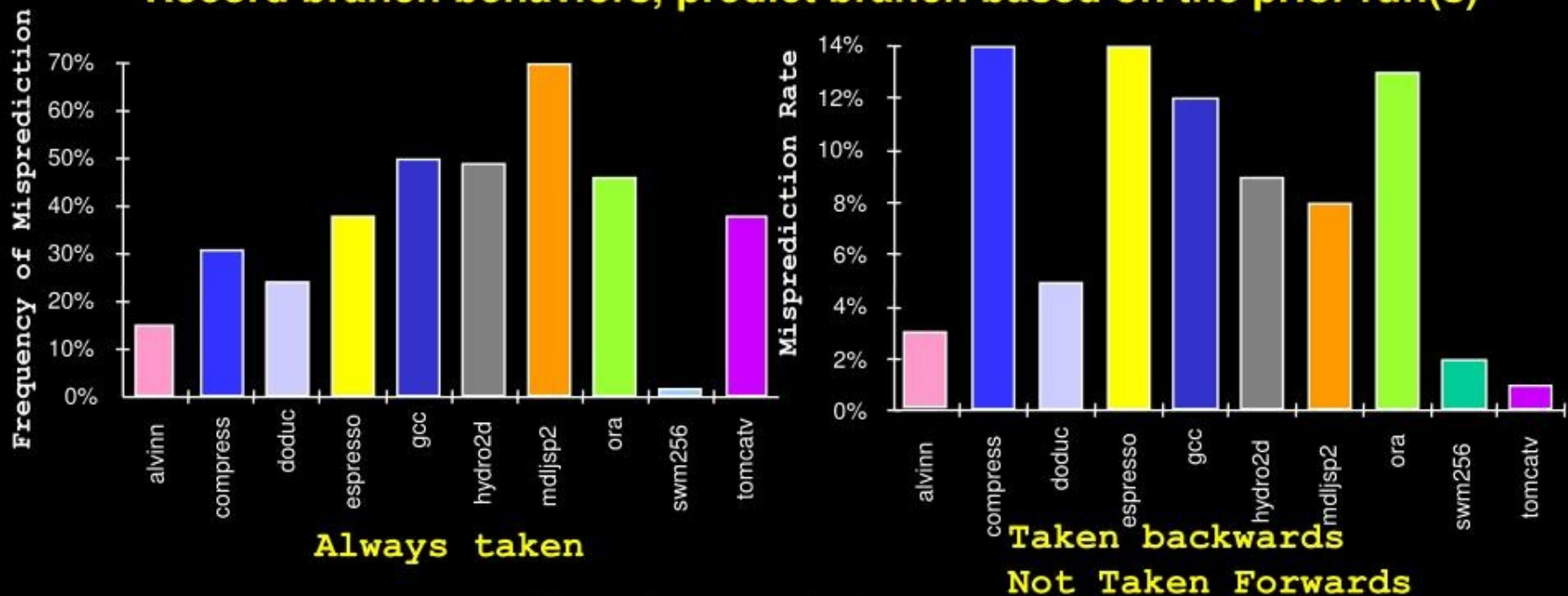
Depend
on LW,
need to
stall

If branch is almost always **NOT TAKEN**,
and R4 is not needed on the taken path,
and R5 and R6 are not modified in the
following instruction(s), this move can
increase speed

If branch is almost always **TAKEN**,
and R7 is not needed, and R8 and R9
are not modified on the fall-through
path, this move can increase speed

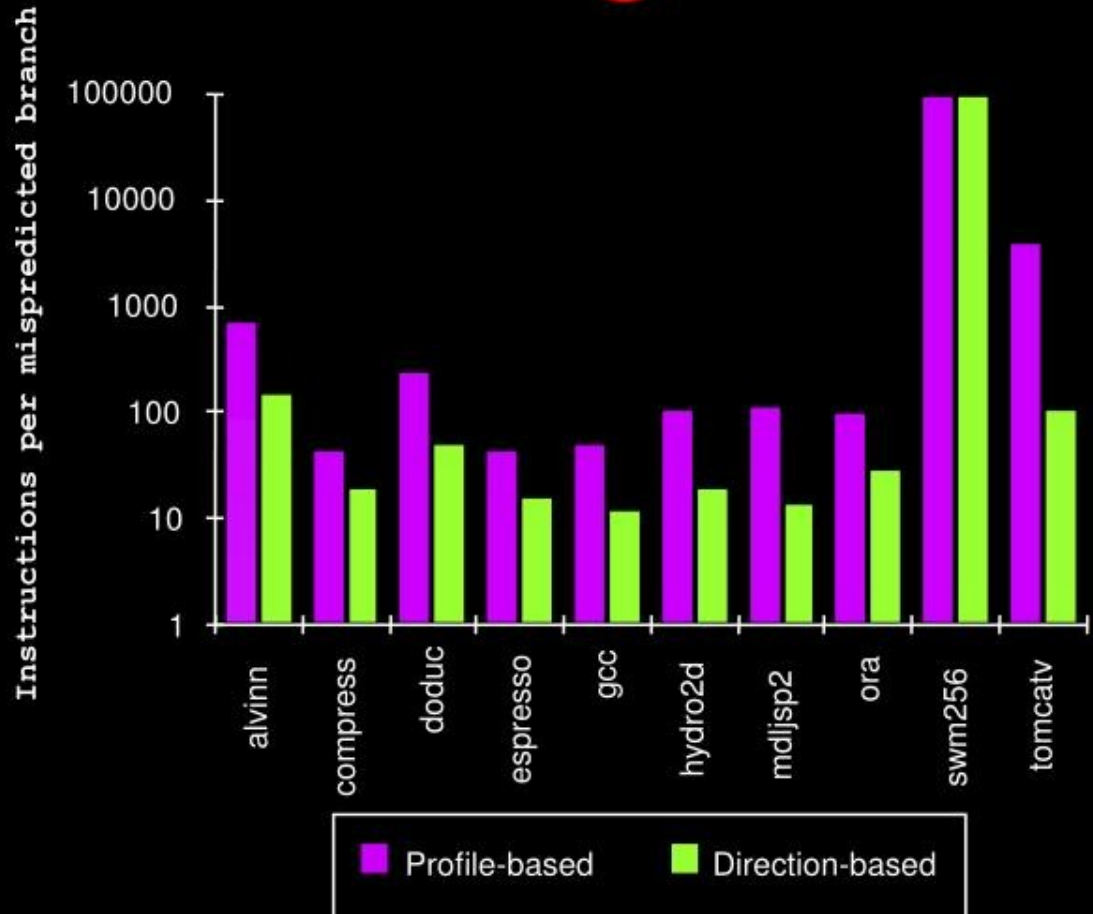
Static(Compiler) Prediction of Taken/Untaken Branches

- Improves strategy for placing instructions in delay slot
- Two strategies
 - Direction-based Prediction:
TAKEN backward branch, **NOT TAKEN** forward branch
 - Profile-based prediction:
Record branch behaviors, predict branch based on the prior run(s)



Evaluating Static Branch Prediction Strategies

- **Misprediction rate ignores frequency of branch**
- **Instructions between mispredicted branches is a better metric**



Pipelining Summary

- Just overlap tasks, and easy if tasks are independent
- Speed Up $\leq$ Pipeline Depth; if ideal CPI is 1, then:

$$\text{Speedup} = \frac{\text{Pipeline Depth}}{1 + \text{Pipeline stall CPI}} \times \frac{\text{Clock Cycle}_{\text{Unpipelined}}}{\text{Clock Cycle}_{\text{Pipelined}}}$$

- Hazards limit performance on computers:
 - Structural: need more HW resources
 - Data: need forwarding, compiler scheduling
 - Control: Dynamic Prediction, Delayed branch slot, Static(compiler) Prediction